

CS781 – AI for Health Sciences

Course Project Milestone 2: Research Progress Checkpoint

Anton Rasmussen

Spring 2026 | Update Date: April 6, 2026

1. Project Title and Team

Beyond Accuracy Loss: Measuring and Recovering Calibration Degradation in a Quantized Biomedical Large Language Model

- **Author:** Anton Rasmussen (sole contributor)
- **Course:** CS781 – AI for Health Sciences, Spring 2026
- **Repository:** [antonrasmussen/cs781-s26](https://github.com/antonrasmussen/cs781-s26), `main` branch (26 commits through 2026-03-24)
- **Project package:** `reliability_eval` (Python, under `project/src/`)

2. Research Goal

This project investigates whether post-training quantization of BioMistral-7B degrades calibration (measured by Expected Calibration Error, ECE) more than classification accuracy (measured by macro F1), and whether standard post-hoc calibration methods can recover the lost calibration. The work targets two widely used biomedical classification benchmarks — PubMed 20k RCT (5-class sequential sentence classification) and MedNLI (3-class natural language inference, accessed via PhysioNet) — evaluated across three precision levels (FP16, INT8, INT4) using the bitsandbytes quantization backend.

Three testable hypotheses:

1. **Primary:** INT4 quantization degrades ECE by a greater relative magnitude than it degrades macro F1 ($|\Delta_{ECE}| > |\Delta_{F1}|$), tested via paired bootstrap comparison.
2. **Secondary:** Temperature scaling applied after quantization recovers ECE to within 110% of the FP16 baseline without materially reducing accuracy.
3. **Tertiary:** INT4 quantization increases prompt sensitivity (decreased inter-template agreement via Fleiss' kappa), and this effect is *not* recovered by post-hoc calibration.

The core motivation is that existing evaluations of quantized biomedical LLMs focus almost exclusively on aggregate accuracy, which is insufficient for safety-critical clinical deployment. This project extends prior observations that quantization degrades calibration by testing whether post-hoc methods can *recover* it — and at what cost to classification accuracy and prompt stability.

3. Progress Update and Evidence

3.1 Completed Components

The following table lists completed components with representative evidence files and their connection to the research goal.

Component	Evidence (file : function)	Research-Goal Link
Project scaffold and documentation	docs/proposal.md, architecture.md, experiment_protocol.md, CONSISTENCY_MATRIX.md, INVENTORY.md	Reproducible experimental design; audit trail for documentation-code consistency
Hierarchical YAML config system	src/reliability_eval/config/resolve.py : resolve_config, resolve_mvp_config	Enables the 30–60 run sweep matrix without manual config editing
CLI and 3 experiment runners	src/reliability_eval/cli.py : _cmd_run, _cmd_sweep; experiments/run_mvp.py, run_local.py, run_grid.py	Multiple execution paths (local, grid, Flyte) for different compute environments
Artifact I/O	src/reliability_eval/io/artifact_store.py : ensure_run_dir, save_predictions, save_metrics; io/run_manifest.py : create_manifest	Structured provenance for every experiment run
Classification metrics	src/reliability_eval/metrics/classification.py : compute_metrics	Macro F1 is the denominator in Hypothesis 1 (Δ_{F1})
ECE and reliability diagrams	src/reliability_eval/metrics/calibration.py : expected_calibration_error, reliability_bins; reporting/reliability_diagrams.py : plot_reliability	ECE is the primary metric for Hypothesis 1; reliability diagrams are the primary visualization
Prompt rendering and label-code mappings	src/reliability_eval/prompting/render.py : render; prompting/label_codes.py : get_label_codes	Single-letter class codes are the intended probability extraction method (proposal Section 5.2); single-token mapping must be verified with the BioMistral tokenizer
Mock inference pipeline	src/reliability_eval/inference/score_class_codes.py : mock_score_example	Validates the pipeline end-to-end without GPU; reduces integration risk for real inference
PubMed RCT data loader	src/reliability_eval/data/pubmed_rct.py : load_pubmed_rct	Data adapter for the primary benchmark task

Component	Evidence (file : function)	Research-Goal Link
Fleiss' kappa (prompt stability)	<code>src/reliability_eval/metrics/prompt_stability.py : fleiss_kappa</code> (fully implemented with edge-case handling)	Core metric for Hypothesis 3 (prompt robustness under quantization)
Temperature scaling (apply + fit)	<code>src/reliability_eval/calibration/temperature_scaling.py : apply_temperature_scaling, fit_temperature</code> (64-point geometric grid search)	The intervention tested in Hypothesis 2 (calibration recovery)
Isotonic calibration (apply)	<code>src/reliability_eval/calibration/isotonic.py : isotonic_calibration, _apply_monotone_then_renorm</code>	Alternative calibration method for the recovery analysis
Calibration dispatch	<code>src/reliability_eval/calibration/apply.py : apply_calibration</code>	Routes to temperature/isotonic/none based on config
Sweep grid expansion	<code>src/reliability_eval/experiments/run_grid.py : expand_sweep</code>	Generates the Cartesian product of precision × dataset × template × calibration
Flyte orchestration wrappers	<code>src/reliability_eval/flyte/tasks.py, flyte/workflows.py</code>	Optional reproducible workflow orchestration
Test suite (10 modules, 38 tests)	<code>tests/test_mvp_runner.py, test_config_resolution.py, test_calibration_apply.py, test_calibration_metrics.py, test_probability_extraction.py, test_prompt_rendering.py, test_prompt_stability.py, test_dataset_registry.py, test_label_code_tokenization.py, test_artifact_store.py</code>	Validates correctness of metrics, config resolution, calibration, and pipeline integration

The mock inference pipeline validates the full data flow end-to-end: config resolution → data loading → prompt rendering → scoring → metrics computation → artifact writing → reliability diagram generation. Once real model inference is connected, the experimental sweep should run through the existing infrastructure without major additional plumbing.

Progress since the March 24 update: The prior update document (`project_update.md`) lists Fleiss' kappa, temperature scaling, and isotonic calibration as "not started." All three are now implemented and tested. This Milestone 2 document reflects the actual code state as of April 6, 2026.

3.2 Overall Project Status

By Milestone 2, the experiment infrastructure is largely ready, but no real model results have been produced yet because model loading and GPU-backed inference remain the main blocking dependency. The project is currently more developed as an evaluation scaffold than as an experimental study — the measurement infrastructure is in place, but it has not yet been exercised on real model outputs. The table below summarizes where each research phase stands:

Research Phase	Status	Detail
Infrastructure (config, CLI, metrics, calibration, artifact I/O, tests)	Complete	16 components implemented and tested; end-to-end pipeline validated with mock inference
Experiments (real model loading, inference, full dataset, prompt sweep)	Not started	Blocked on BioMistral-7B loading + GPU access; remaining stubs depend on the same model-loading implementation
Analysis (hypothesis testing, bootstrap CIs, reporting, final paper)	Not started	Depends on experiment outputs; analysis code (ECE, Fleiss' kappa, temperature scaling) is ready

3.3 Partially Complete

Component	What Exists	What Is Missing
PubMed data adapter	Tiny 8-example JSONL sample (<code>data/samples/pubmed_rct_tiny.jsonl</code>) + loader	Full HuggingFace dataset integration; real train/val/calib/test splits
Dataset splits	<code>src/reliability_eval/data/splits.py : get_splits</code> returns hardcoded list <code>["train", "val", "calibration", "test"]</code>	Actual split logic that partitions loaded data into those splits
Prompt templates	2 PubMed templates (<code>pubmed_t1</code> , <code>pubmed_t2</code>)	3 more PubMed + 5 MedNLI templates (proposal requires 5 per task)
Isotonic calibration	Apply + renormalize implemented	<code>fit_isotonic</code> raises <code>NotImplementedError</code>
CLI	<code>run</code> and <code>sweep</code> subcommands work	<code>report</code> subcommand prints "not yet implemented"

3.4 Not Started

Real inference integration: The following stubs all depend on BioMistral-7B model loading. Rather than independent gaps, they form a single blocking dependency:

- Model loading at FP16/INT8/INT4: `src/reliability_eval/models/load_model.py`, `models/quantization.py`, `models/tokenizer_utils.py`
- Real single-token class-code logit extraction: `src/reliability_eval/inference/score_class_codes.py` : `score_example`
- Batch runner: `src/reliability_eval/inference/batch_runner.py` : `run_eval`

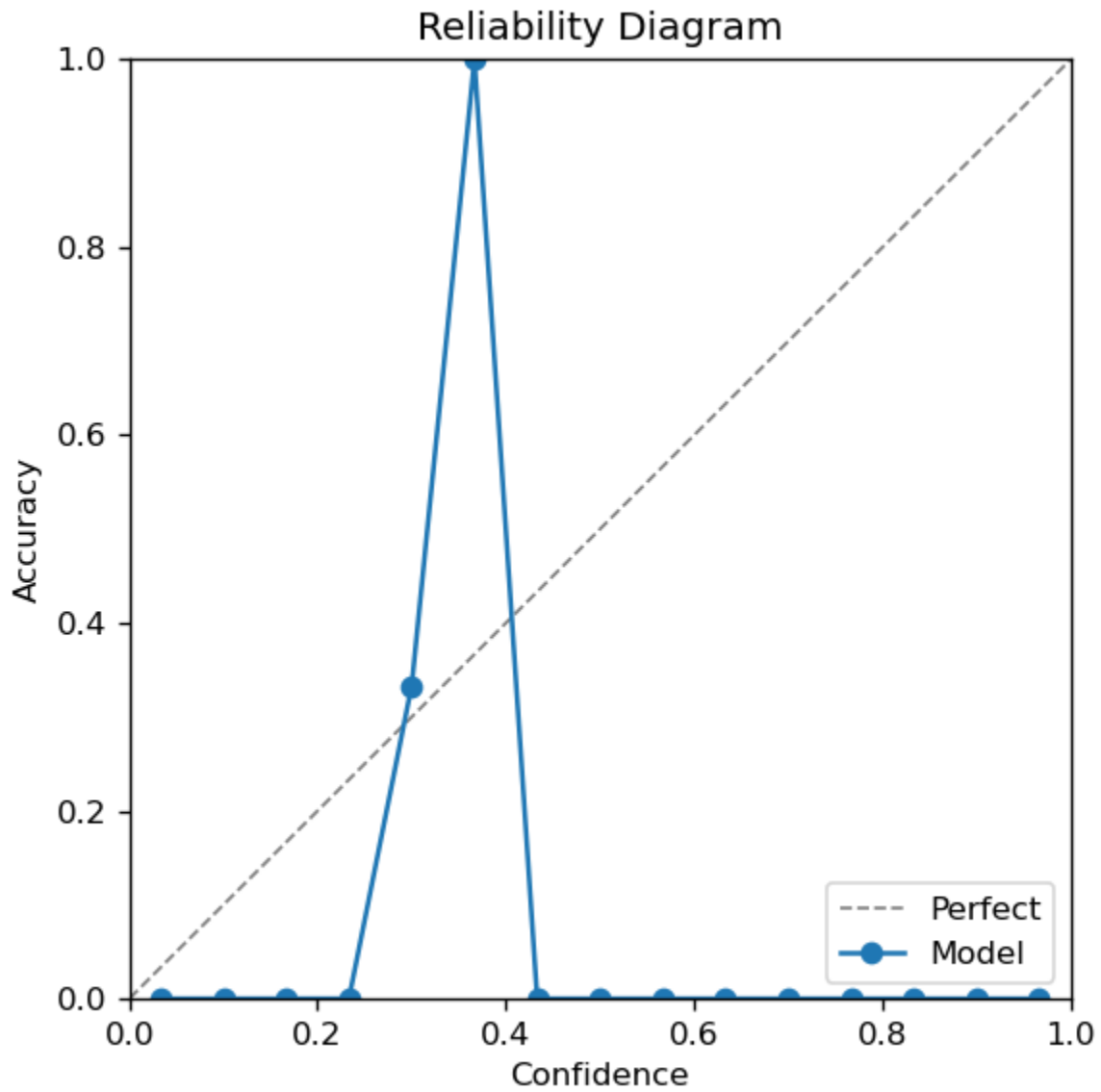
Downstream items (depend on real inference):

- MedNLI dataset loader (`src/reliability_eval/data/mednli.py` — contingent on PhysioNet access; proposal fallback is PubMed-only)
- ACE (adaptive calibration error), bootstrap confidence intervals, per-sample flip rate
- Cross-run aggregation (`src/reliability_eval/experiments/aggregate.py`)
- Reporting tables and export (`src/reliability_eval/reporting/tables.py`, `reporting/export_summary.py`)
- Efficiency metrics (`src/reliability_eval/metrics/efficiency.py`)

3.5 Pipeline Validation Artifacts

The following artifacts were generated on April 6, 2026 by running the MVP pipeline (`experiments/run_mvp.py`) with mock inference on the 8-example PubMed RCT tiny sample. They demonstrate that the pipeline functions correctly end-to-end. **These are synthetic outputs from the deterministic mock scorer and do not represent real model performance.**

Reliability Diagram



Generated from mock inference on 8-example PubMed RCT sample. Demonstrates that ECE computation, binning, and plot generation function correctly end-to-end.

Metrics Output

```
{
  "accuracy": 0.375,
  "macro_f1": 0.26,
  "per_class_f1": {
    "BACKGROUND": 0.0,
    "CONCLUSIONS": 0.5,
    "METHODS": 0.0,
    "OBJECTIVE": 0.0,
    "RESULTS": 0.8
  },
  "n_examples": 8,
  "ece": 0.13724415660881062
}
```

Metrics output from mock inference. Values are synthetic (mock scorer artificially inflates confidence for the correct label by +0.35) and do not represent model performance.

Test Suite

```
===== 37 passed, 1 xfailed in 19.10s =====
```

All 37 tests pass across 10 test modules covering config resolution, metrics computation, calibration, prompt rendering, probability extraction, artifact I/O, and end-to-end pipeline integration. The single `xfail` is a placeholder test (`test_tokenization_single_token`) that verifies single-letter class codes map to exactly one token ID in the real BioMistral tokenizer — it cannot pass until the actual model tokenizer is loaded, which is expected.

4. Choke Point Analysis

4.1 Primary Bottleneck: Real Model Loading and Inference

The entire experimental pipeline — from FP16 baselines through quantization experiments and calibration recovery — is blocked on a single dependency: implementing real model loading and single-token logit extraction in the `models/` and `inference/` subpackages.

Three functions must be implemented:

1. `src/reliability_eval/models/load_model.py : load_biomistral` — load BioMistral-7B at FP16, INT8, and INT4 precision levels

2. `src/reliability_eval/models/tokenizer_utils.py : get_code_token_ids` — map class-code letters (A–E) to their token IDs
3. `src/reliability_eval/inference/score_class_codes.py : score_example` — extract logits at the first generated token position, select code-token logits, compute restricted softmax

Every module downstream — calibration recovery, prompt stability across real templates, bootstrap CIs, reporting — depends on having *real* probability distributions. The mock inference pipeline has validated the plumbing, but produces synthetic distributions that cannot test the research hypotheses.

4.2 Why This Is Technically Non-Trivial

This is not simply "code that hasn't been written yet." Four specific technical risks make the implementation uncertain:

Tokenizer behavior. The project assumes that single-letter codes (A, B, C, D, E) each map to exactly one token ID, but this must be verified empirically for the BioMistral tokenizer before experiments proceed. BioMistral extends the base Mistral tokenizer with biomedical vocabulary; there is a risk that the tokenizer treats " A" (with leading space) differently from "A", or that the extended vocabulary introduces unexpected token splits. The `test_label_code_tokenization.py` test has an `xfail` placeholder for exactly this verification, which cannot pass until the real tokenizer is loaded.

Logit extraction position. For a decoder-only model with a prompt followed by a completion cue, the logits at the *correct token position* must be extracted. Off-by-one errors in the position index would silently produce nonsense probability distributions. The mock scorer sidesteps this entirely by generating probabilities directly; the real implementation must correctly identify the first generated token position after the prompt.

Quantization configuration. `bitsandbytes` INT4 (NF4) with double quantization requires specific `BitsAndBytesConfig` parameters. Incorrect configuration can silently fall back to FP16, producing misleading results that appear to show "no quantization effect." The quantization module must verify the model is actually loaded at the intended precision by checking weight dtypes or memory footprint.

Hardware dependency. As a rough weights-only estimate, a 7B model requires about 14 GB at FP16 and about 3.5 GB at 4-bit precision, though actual inference memory depends on KV cache, activations, and framework overhead. The project needs access to a GPU with at least 16 GB VRAM. The current compute options are ODU HPC nodes or a cloud GPU (Colab Pro, Lambda, etc.), neither of which is configured yet.

4.3 Secondary Bottleneck: Full Dataset Integration

The PubMed loader currently uses an 8-example fixture (`data/samples/pubmed_rct_tiny.jsonl`). Real experiments need the full 20k-abstract PubMed RCT dataset loaded from HuggingFace, with proper train/val/calibration/test split logic implemented in `src/reliability_eval/data/splits.py` (currently a stub returning hardcoded split names).

4.4 Timeline Context

The proposal's Phase 3 (quantization experiments) was scheduled to end April 6 — today. The project is approximately three weeks behind the original timeline. This delay reflects a deliberate trade-off: the scaffold phase was more extensive than initially planned — building a full config composition system, CLI tooling, Flyte wrappers, a 10-module test suite, and a documentation audit trail. I prioritized infrastructure and testing early, which should reduce integration risk when real inference is connected, but it means the first real model run has not happened yet. In retrospect, the time spent on reproducibility infrastructure was probably worthwhile, but it did push back the start of actual experiments further than I expected.

5. To-Do List for the Next Phase

Three steps bridge the gap between the current state (validated mock pipeline) and the research goal (tested hypotheses with real model outputs), aligned to remaining deadlines.

Step 1: Real Model Loading and Probability Extraction

Target: April 13 (before Milestone 3)

- Implement `load_model.py` : `load_biomistral` for FP16, INT8, INT4 using `transformers` + `bitsandbytes`
- Implement `tokenizer_utils.py` : `get_code_token_ids` to map class-code letters to token IDs
- Implement `score_class_codes.py` : `score_example` for single-token logit extraction + restricted softmax
- Pass the proposal's validation gate (Section 5.2): on 200 held-out examples, verify probability normalization, argmax consistency with greedy decode, and visual plausibility of FP16 reliability diagrams
- **Dependency:** Secure GPU access (ODU or cloud). Add `torch`, `transformers`, `bitsandbytes`, `accelerate` to `requirements.txt` (currently listed as TODO)

Step 2: Full Data Pipeline and Prompt Expansion

Target: April 20 (Milestone 3 delivery)

- Integrate PubMed 20k RCT via HuggingFace `datasets` library; implement real train/val/calib/test splits in `data/splits.py`
- Expand PubMed prompt templates from 2 to 5 (meaning-preserving variation per proposal Section 5.3.3)
- Validate all 5 templates on 50 dev examples at FP16 (template validation gate)
- If PhysioNet access is confirmed, implement `data/mednli.py` loader and 5 MedNLI templates; otherwise invoke the proposal fallback (PubMed-only)
- Implement `fit_isotonic` in `calibration/isotonic.py`
- **Milestone 3 target:** FP16 baselines complete on PubMed RCT; at least INT8 evaluated

Step 3: Full Experimental Sweep and Statistical Analysis

Target: May 1 (3 days before final submission)

- Execute the 3 precision $\times$ 1–2 tasks $\times$ 5 templates $\times$ 2 calibration states = 30–60 run matrix via `run_grid.py`
- Implement bootstrap CIs ($n=1000$) for all classification and calibration metrics
- Implement ACE (adaptive calibration error, equal-mass bins)
- Implement per-sample flip rate alongside the existing Fleiss' kappa
- Implement `aggregate_metrics` for cross-run synthesis
- Test the three hypotheses with paired bootstrap comparisons
- Generate publication-quality reliability diagrams, calibration recovery plots, and efficiency summary table
- Write IEEE-format final report (May 1–4)

Fallback Plan

I am not fully confident that GPU access and model loading can be resolved within one week, so the fallback plan below is a realistic contingency rather than a formality.

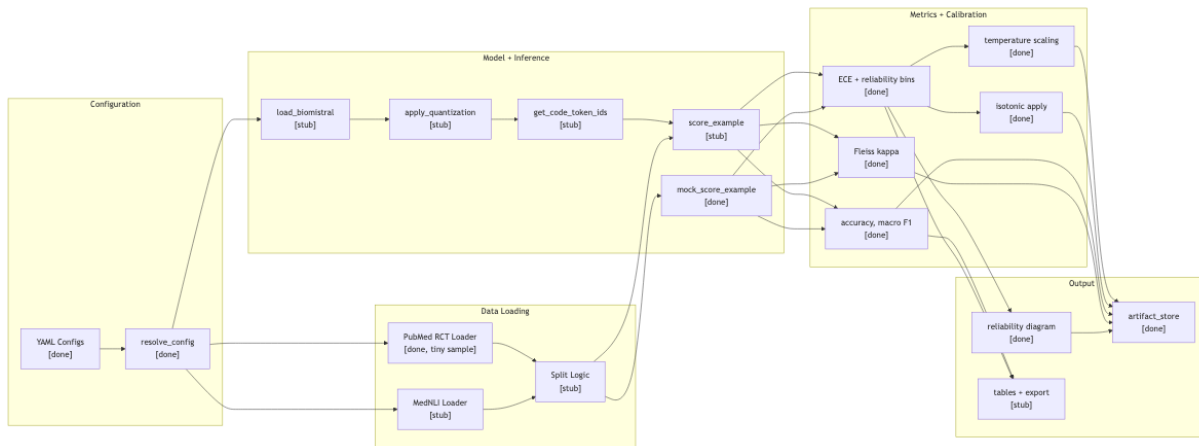
Decision rule: If FP16 real inference is not working by April 13, the project will formally narrow to PubMed-only and INT8-first execution. Specifically:

- **April 13 checkpoint — no working FP16 inference:** Narrow scope to PubMed RCT only (drop MedNLI), halving the experiment matrix. Allocate all remaining GPU-setup effort to a single precision level (INT8) before attempting INT4.

- **April 18 checkpoint — no working INT8 inference:** Escalate to cloud GPU (Colab Pro or Lambda) with a 48-hour time-box. If that also fails, document the validated pipeline and implementation as the main result of the project, with real inference results presented as future work.
- **INT4 technical failure at any point:** The INT8 results would still support a narrower but viable analysis. INT4 is a stretch goal rather than a requirement.

6. Experiment Pipeline Architecture

The following diagram shows the end-to-end pipeline. Nodes marked with **[done]** are implemented and tested; nodes marked with **[stub]** are not yet implemented.



The main dependency chain runs through `LoadModel` → `Quantize` → `Tokenizer` → `Score`. Once this chain is implemented, all downstream metrics and output nodes already function correctly (validated via the `MockScore` path).

7. Questions for Peer Review

I would appreciate feedback on the following architectural and methodological decisions:

1. **Probability extraction validity.** My approach extracts class probabilities by taking the softmax over only the logits of 5 (or 3) single-letter code tokens, discarding the rest of the vocabulary. Could this restricted softmax introduce systematic bias in the probability estimates compared to the full-vocabulary distribution, and if so, how would you control for that?
2. **Calibration measurement under distribution shift.** Temperature scaling is fitted on a held-out calibration split (15% of validation data) and evaluated on the test set. Given that quantization may shift the model's internal representations, is it valid to fit the

calibration method on quantized-model outputs and evaluate on the same quantized model — or should the calibration set also be used to measure the *shift* relative to FP16 as a reference?

3. **Prompt stability as an independent failure mode.** The tertiary hypothesis claims that prompt instability under quantization is independent of calibration degradation. What additional evidence — beyond showing that temperature scaling does not recover Fleiss' kappa — would you need to convincingly argue these are genuinely separate failure modes rather than correlated symptoms of the same underlying representational damage?

References

- Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS 2022*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS 2023*.
- Dernoncourt, F., & Lee, J.Y. (2017). PubMed 200k RCT: a Dataset for Sequential Sentence Classification in Medical Abstracts. *IJCNLP 2017*.
- Fleiss, J.L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 378–382.
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K.Q. (2017). On Calibration of Modern Neural Networks. *ICML 2017*.
- Labrak, Y., Bazoge, A., Morin, E., Gourraud, P.-A., Rouvier, M., & Dufour, R. (2024). BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains. *ACL 2024 Findings*.
- Romanov, A., & Shivade, C. (2018). Lessons from Natural Language Inference in the Clinical Domain. *EMNLP 2018*.
- Zadrozny, B., & Elkan, C. (2002). Transforming Classifier Scores into Accurate Multiclass Probability Estimates. *KDD 2002*.